FleetWorks Application Architecture Design

Prepared on: 2026-08-01

1. Executive Summary

FleetWorks is a static, browser-first fleet management application backed by Supabase. The application is built with multi-page HTML, shared CSS, and vanilla JavaScript modules that run directly in the browser. Supabase provides the production backend capabilities: authentication, Postgres database, Row Level Security, REST/RPC APIs, private file storage, and Edge Functions.

> Current architecture style: static multi-page SaaS/PWA + browser-side business logic + Supabase backend-as-a-service. There is no traditional application server for business logic. The local Node server only serves static files for development and testing.

2. Layer-Wise System Architecture

USER CHANNELS

Fleet Ownerfleet.html

index.html

dashboard redirects

Supervisor / Driverteam.html

driver.html no-login links

Mechanic / Garagegarage.html

partner.html

Admin / Staffadmin.html

signin.html

V

PRESENTATION LAYER

Static HTMLindex, fleet, team, driver, garage, admin, partner, signin, reset

Stylingcss/style.css

css/landing.css

Assetslogos, icons, manifest, service worker

HostingStatic hosting in production

Node static server locally

V

CLIENT APPLICATION LAYER

Core Appfleet.js

dbcore.js

cloudstore.js

Operationsworkflow.js

team.js

driver.js

garage.js

Financepayroll.js

analytics.js

bulkimport.js

Assistantscopilot.js

fleetmap.js

account.js

V

CLIENT STATE AND OFFLINE LAYER

localStorageff_fleet

ff_fleet:<user>

fw_service_requests

Session Cachefw_session:<user>

fw_session:active

Demo Mode Local data for exploration without login

Hybrid Sync Core data DB-direct

settings blob-backed

V

SUPABASE API LAYER

Auth API Signup, login, recovery, token refresh

PostgRESTTable CRUD and RPC from browser modules

Storage API Private bills bucket and signed URLs

Edge Functions Privileged and secret-bearing operations

v

DATA, SECURITY, AND DOMAIN LAYER

Tenant Coreorganizations

memberships

Fleet Corevehicles, drivers, expenses, fuel_logs, issues, work_orders

Operationsreminders, inspections, parts, documents, tyre_readings, trips

SecurityPostgres RLS policies are the main authorization boundary

v

EXTERNAL SERVICE LAYER

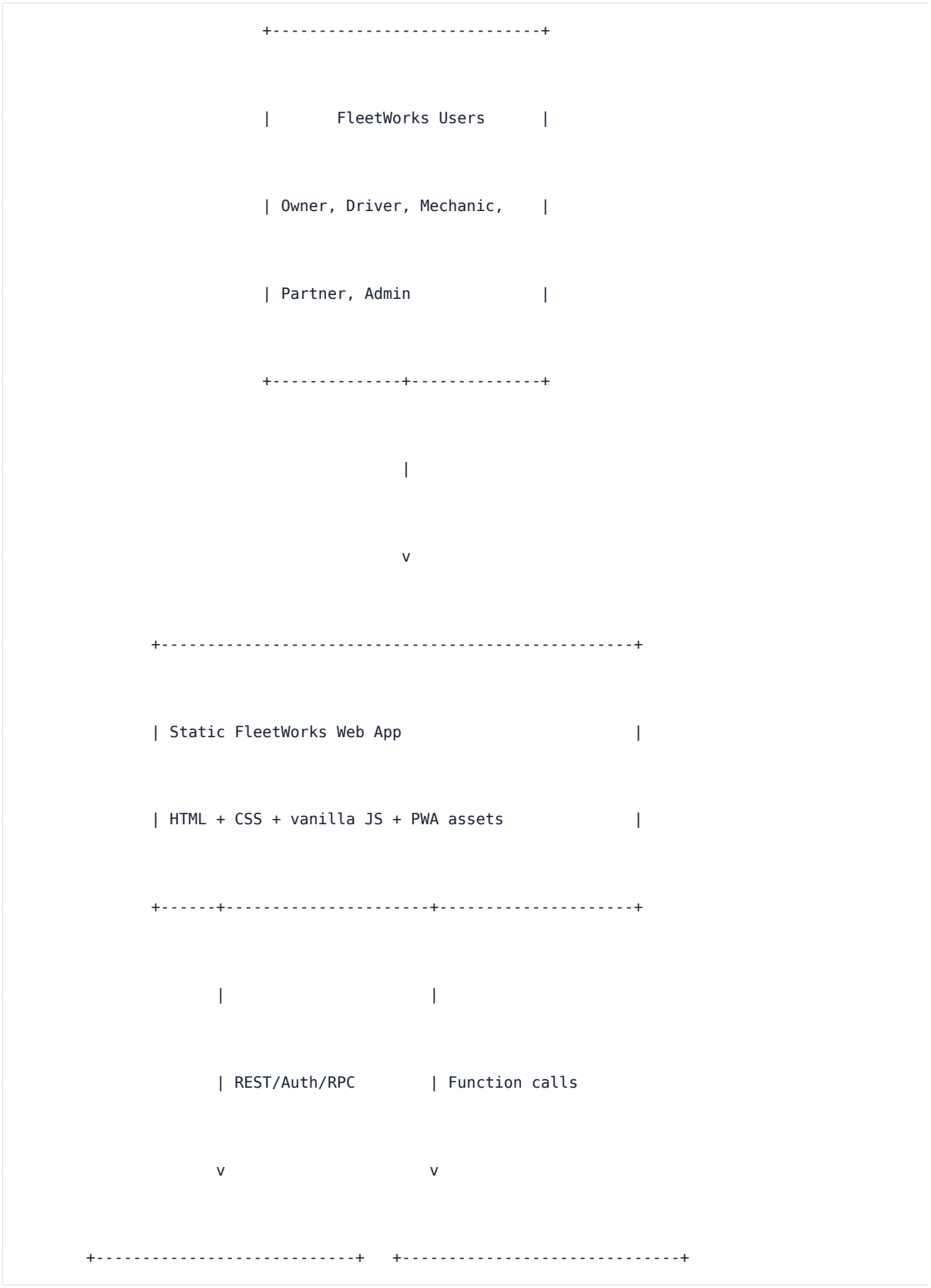
Anthropic Claude Copilot Edge Function

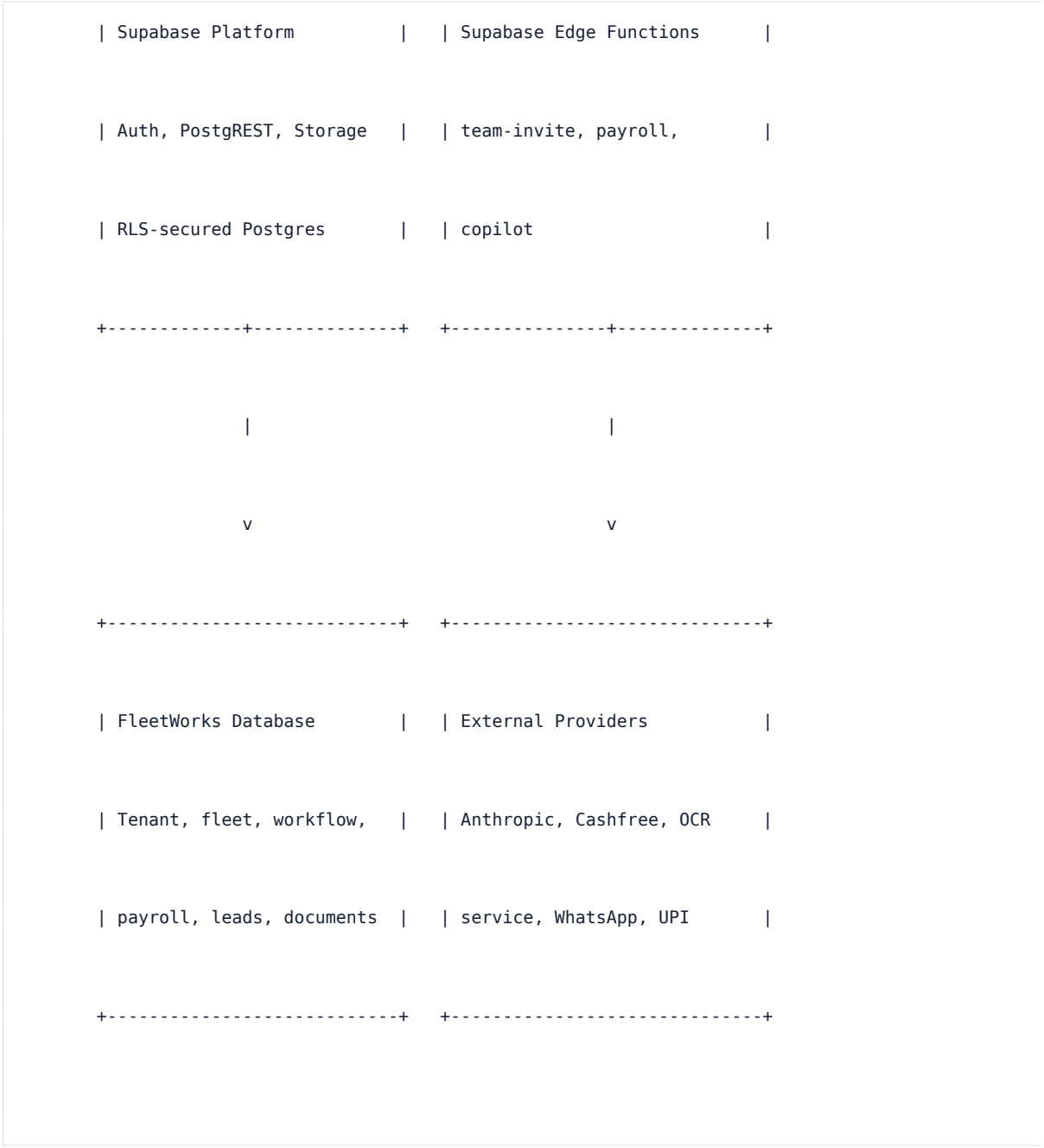
CashfreePayroll payout functions

PaddleOCROptional FastAPI OCR service

Client Links	WhatsApp and UPI deep links

3. Typical System Context Diagram





4. Runtime And Deployment Architecture

5. Module Breakdown

Area	Files	Responsibilities
Owner Fleet Manage	fleet.html, js/fleet.js, js/dbcore.js, js/cloudstore.js	Auth gate, fleet forms, render orchestration, normalized DB CRUD, local/demo state.

r

Analytics and Reports	js/analytics.js, js/copilot.js, js/fleetmap.js	FleetIQ views, assistant, summaries, map-related views.
Team Portal	team.html, js/team.js	Supervisor/driver login, RLS-scoped vehicle access, fuel/issues/inspection updates, expense requests.
No-Login Driver Link	driver.html, js/driver.js	Tokenized driver submissions into driver_entries through anonymous insert.
Garage Workflow	garage.html, js/garage.js, js/workflow.js	Owner-mechanic service request lifecycle and local/cloud workflow state.
Payroll	js/payroll.js, supabase/functions/payroll-*	Manual salary logs, payment requests, Cashfree payout setup and transfer flow.
Public and Admin	index.html, partner.html, admin.html, signin.html	Lead capture, vendor applications, staff/admin workflows.

6. Data Architecture

Tenant Model

```
auth.users

    |

    v

memberships ----> organizations

    |
```

```
    +-- role: owner, manager, viewer, supervisor, driver
```

Core Fleet Entity Model

```
organizations

    |

    +-- vehicles

    |     |

    |     +-- fuel_logs

    |     +-- expenses

    |     +-- issues

    |     |     |

    |     |     +-- work_orders

    |     |

    |     +-- reminders

    |     +-- inspections

    |     +-- tyre_readings

    |     +-- documents

    |     +-- trips
```

```
    |

    +-- drivers

    |     |

    |     +-- documents

    |     +-- driver_ledger

    |     +-- driver_payout_details

    |

    +-- parts

    +-- memberships

    +-- vehicle_assignments
```

Hybrid Storage Model

```
Signed out:

  Browser localStorage only



Signed in:

  Core entities:

    vehicles, drivers, expenses, fuel_logs, issues, work_orders,
```

```
    reminders, inspections, parts, documents, tyre_readings, trips,

    driver_ledger

    -> Supabase normalized tables



Settings/demo metadata:

    -> localStorage + fleets.data blob

    -> projected into organizations by sync_fleet_from_blob
```

7. Key Data Flows

Flow	Path
Owner sign-in	fleet.html -> cloudstore.js login -> Supabase Auth -> pull settings blob -> dbcore resolves org -> load normalized tables -> renderAll.
Owner saves core record	Form submit -> fleet.js -> dbcore mapper -> fwCloud REST helper -> PostgREST -> RLS -> Postgres -> local db update -> render affected panels.
Team update	team.html -> login -> membership and assignments -> RLS-scoped vehicle query -> direct writes for fuel/issues -> expense_change_requests for expenses.
No-login driver submission	driver.html URL token -> anonymous insert into driver_entries -> owner imports and merges later.
Copilot question	copilot.js -> compact fleet summary -> Supabase Edge Function -> Anthropic -> response to browser, with rule-based fallback.

8. Security Architecture

- The Supabase publishable anon key is public by design.
- Supabase RLS is the primary authorization boundary.

- Authenticated browser requests use the signed-in user's JWT.
- Service role access exists only in Edge Functions.
- Anthropic and Cashfree secrets stay server-side.
- Storage files are private and scoped by user id path.
- Team users see only assigned vehicles through database policy, not client-side filtering.
- No-login driver links can insert into limited intake tables only.

9. Strengths

- Simple static deployment with low operational overhead.
- Offline/demo fallback through localStorage.
- Clear multi-tenant boundary through organizations and memberships.
- Normalized DB-direct data path for core operational records.
- RLS-centered security model.
- Edge Functions isolate privileged writes and integration secrets.

10. Risks And Improvement Areas

- Browser-global script loading order is part of the runtime contract.
- fleet.js carries many responsibilities and can become hard to maintain.
- The shared mutable global db object can make state changes difficult to trace.
- Some behavior remains hybrid between normalized tables, localStorage, and blob sync.
- Direct PostgREST calls from many modules can duplicate mapping and error handling.

11. Recommended Evolution

Horizon	Recommendation
Short term	Keep static/Supabase architecture, document feature ownership, continue moving production records to normalized tables, keep authorization in RLS.
Medium term	Extract domain-specific data-access modules, split fleet.js by domain, add a single state facade around db, expand RLS-driven tests.
Long term	Consider a component framework or backend-for-frontend only if UI/state orchestration outgrows the current static model.

12. Source Files Referenced

package.json, server/static-server.mjs, fleet.html, team.html, driver.html, garage.html, js/backend.js, js/cloudstore.js, js/dbcore.js, js/fleet.js, js/team.js, js/driver.js, js/workflow.js, js/payroll.js, js/copilot.js, supabase/migrations/*.sql,

supabase/functions/*/index.ts, server/ocr/app.py